Semana 8

Desarrollo de Aplicaciones Móviles (DSY2204)

Formato de respuesta

Nombre estudiante:	Pablo Fuentes
Asignatura: Desarrollo de Aplicaciones Móviles (DSY2204)	Carrera: Ingeniería en Desarrollo de Software
Profesor: Miguel Puebla	Fecha: 05-10-2026

Descripción de la actividad

En esta octava semana, se realizará la actividad sumativa 3, llamada " Preparación de una aplicación móvil Android/Kotlin para su distribución". Deberán integrar a la app creada en el framework Android Studio y Kotlin el front end y back end, y preparar la aplicación móvil para su publicación en una distribución de descarga de app. Entrega de encargo con documentación del desarrollo y código del proyecto comprimido en .zip.

Instrucciones específicas

A continuación, se detallan las instrucciones y requisitos de la actividad sumativa 2, deberás leer cada uno de ellos y apuntar al cumplimiento.

Características del producto:

- Integrar en la aplicación móvil de accesibilidad para personas con discapacidad sensorial auditiva desarrollada en la experiencia de aprendizaje 1, el front end y back end en Kotlin, además preparar y publicar el APK firmado.
- La aplicación móvil deberá ser diseñada con componentes UI específicamente para personas con discapacidad sensorial auditiva, facilitando a los usuarios su comunicación (escribir y hablar) en su entorno cotidiano, mejorando su calidad de vida al transformar sus dispositivos móviles en una herramienta que los integra al mundo real y digital.

Requerimientos generales:

- Para el desarrollo de la entrega solicitada considerar los siguientes aspectos:
- Integrar el front end y backend de la aplicación desarrollada durante la asignatura, utilizando Kotlin y Jetpack Compose y manteniendo las funcionalidades implementadas en las semanas anteriores.
- Integrar la persistencia y autenticación de usuarios, utilizando el servicio o motor seleccionado para la aplicación y considerando el manejo de sesión mediante SharedPreferences.
- Implementar operaciones CRUD para registrar, consultar, modificar y eliminar los datos asociados a las funcionalidades de la aplicación.
- Desarrollar y ejecutar pruebas sobre la aplicación, utilizando al menos una de las herramientas abordadas durante la semana: JUnit, Mockito, Robolectric, Espresso o Firebase Test Lab, según las características de la prueba realizada.
- Generar y firmar el APK de la aplicación, dejándolo preparado para su distribución o publicación en una plataforma de descarga.

Requerimientos técnicos

- Desarrollar la aplicación con el Framework Android Studio.
- Generar y compartir el repositorio Git del proyecto.
- Integrar el lenguaje de programación Kotlin al código fuente de la app móvil.
- Integrar los componentes UI de Android Studio y Kotlin.
- Integrar Firebase para gestionar la persistencia de los datos y la autorización a ellos.
- Desarrollar pruebas unitarias en Kotlin.
- Generar el APK.
- Firmar y publicar el APK de la aplicación móvil desarrollada.

Entrega de encargo con documentación

Esta actividad tendrá dos partes y se entregará todo en **un solo ZIP.**
En la primera parte, tras finalizar el desarrollo del proyecto desde Android Studio, se deberá comprimir la carpeta del código del proyecto completo en formato .zip desde Git, además de generar, firmar y publicar el APK de la app y adjuntarlo en espacio indicado en el AVA.
En la segunda parte, se deberá documentar todo el proceso de desarrollo del proyecto, describiendo el rendimiento de las views diseñadas y de las pruebas unitarias, las restricciones y describir el seguimiento de los 5 riesgos técnicos del proyecto definidos en la experiencia de aprendizaje 1, además de describir los resultados de los accionables de mitigación de cada riesgo definido. Todo ello **en este mismo documento de formato de respuesta, a continuación de estas instrucciones** y subirlo en el ZIP que entregarás con el desarrollo del Proyecto y este documento del formato de respuesta debe subirse en formato PDF.

- Entregar el producto totalmente finalizado, según el avance solicitado en esta entrega.
- No se aceptarán entregas fuera de plazo ni por otro medio que no sea AVA.
- Todos los productos desarrollados con IA serán evaluados con nota mínima.

Restricciones en esta entrega
La aplicación móvil debe ser adaptativa a múltiples dispositivos Android que cuenten con una conexión activa a internet.

Formato del texto:

- Fuente: Arial.
- Tamaño de fuente: 12 puntos.
- Interlineado: 1,5 líneas.
- Alineación del texto: a la izquierda.

Iniciar documentación desde la siguiente página.

Datos del documento

Histórico de revisiones

Versión	Fecha	Descripción/cambio	Autor
0.1	08-09-2026	Base de continuidad y riesgos del proyecto.	Pablo Fuentes
0.9	05-10-2026	Integración de pruebas JUnit y configuración release 1.0.0.	Pablo Fuentes
1.0	05-10-2026	APK firmado, resultados, trazabilidad y publicación documentada.	Pablo Fuentes

Información del proyecto

Organización	Duoc UC · Campus Virtual
Sección	Desarrollo de Aplicaciones Móviles
Proyecto (nombre)	SeñalSegura · Android/Kotlin
Fecha de inicio	Continuidad del proyecto de la asignatura
Fecha de término	05-10-2026
Patrocinador principal	Requerimiento académico
Docente	Miguel Puebla

Alcance del proyecto. *Problema, solución propuesta y contexto del proyecto.*
SeñalSegura atiende barreras de comunicación cotidiana de personas con discapacidad auditiva. Integra una interfaz Android con Kotlin/Jetpack Compose y Material 3, autenticación, persistencia y sincronización por usuario en Firebase Realtime Database. Mantiene un modo local de respaldo para demostración cuando Firebase no está configurado. El cierre de Semana 8 suma pruebas unitarias, configuración de versión, APK Release firmado y verificación criptográfica. La entrega documenta rendimiento de vistas, restricciones y la evolución de los cinco riesgos definidos previamente.

Funciones esperadas del software a desarrollar.

• Acceso, registro y recuperación de cuenta con Firebase Authentication; respaldo de demostración local.
• Inicio: mensaje visual de alto tamaño, análisis de prioridad, frases rápidas y texto a voz del sistema.
• Alertas: consulta, búsqueda, filtros, favoritas, detalle y CRUD asociado al UID.
• Perfil y sesión: Firebase Realtime Database y SharedPreferences en modo privado.
• Accesibilidad: grillas adaptativas, soporte TalkBack, alto contraste, Palette y feedback háptico.
• Integración Android: ContentProvider para alertas, widget de pantalla inicial y navegación Compose.

Rendimiento esperado del software a desarrollar.

Análisis cualitativo del código y la ejecución de pruebas; no se realizó benchmark de FPS, memoria ni latencia en dispositivos físicos.

• Login/Registro/Recuperación: validación local previa al acceso remoto; estado del formulario conservado con rememberSaveable. Firebase usa callbacks; la vista mantiene el control del formulario.
• Inicio: LazyVerticalGrid con celdas adaptativas (mínimo 170 dp); el mensaje principal y sus estadísticas se presentan en pantalla. TTS es inicializado por efecto y el texto permanece como alternativa visible.
• Alertas: LazyVerticalGrid adaptativa (mínimo 230 dp); búsqueda/filtros en estado local y carga del ContentProvider al entrar. Palette se calcula con Dispatchers.Default y colores de respaldo.
• Detalle: reutiliza componentes de alerta, jerarquía tipográfica y contraste legible; la paleta tiene valores fallback.
• Backend/Perfil: LazyColumn y repositorios; lecturas y escrituras Firebase asíncronas. La paginación no es necesaria para el volumen de demostración actual.

La ejecución reportó 40 pruebas JUnit sin fallos. No hubo prueba instrumentada con Espresso ni medición de tiempos de respuesta contra Firebase.

Restricciones del proyecto.
• Android, API mínima 24; compilación y target SDK 36. Requiere conexión a Internet para autenticación y sincronización Firebase. • Herramientas verificadas: JDK 21, Gradle 9.5.0, AGP 9.3.1 y Android Build Tools 36.1.0. • TTS, vibración y servicios de accesibilidad dependen de recursos y ajustes del dispositivo. • El modo local es demostrativo; la sincronización persistente y aislamiento de datos requieren Firebase configurado y sus reglas publicadas. • Pruebas ejecutadas: unitarias JVM; no se verificó matriz de emuladores/dispositivos ni Firebase Test Lab. • Se entrega APK firmado de prueba. Por instrucción docente se documenta Google Play, sin comprar una cuenta ni afirmar publicación en tienda. • La clave privada temporal de firma no se incorpora al repositorio; una publicación futura requiere guardar una clave de carga permanente en un gestor seguro.

Estructura de desglose de trabajo. *Definición de Diagrama y diccionario EDT. Adjuntar diagrama y diccionario.*
El diagrama y diccionario EDT se adjuntan en los Anexos 1 y 2. Paquetes de trabajo: 1.1 interfaz accesible Compose; 1.2 backend Firebase, sesión y CRUD; 1.3 pruebas JUnit, compilación y firma APK; 1.4 documentación, control Git y proceso de distribución. Cada bloque identifica productos verificables en el código y los archivos del proyecto.

Resumen de riesgos

Listado de riesgos relacionado al desarrollo de S.W. Indicar riesgo, etapa o fase en que se presenta, la probabilidad de que ocurra, magnitud o impacto de este riesgo por etapa en el proceso.

***Probabilidad:** alta, media, baja.*
***Impacto:** alto, significativo, moderado, inferior y bajo.*

Riesgo	***Fase***	***Probabilidad***	***Impacto***	***Acción de mitigación***
Incompatibilidad entre SDK, AGP y Gradle	Configuración / release	Media	Alto	Controlado: JDK 21, Gradle 9.5, AGP 9.3.1, SDK 36 y Build Tools 36.1.0 fijados. testDebugUnitTest y assembleRelease finalizaron correctamente.
El emulador no inicia por virtualización o gráficos	Pruebas	Media	Significativo	Residual: compilación y JUnit JVM verificados; esta entrega no ejecutó emulador ni prueba UI instrumentada. Mantener arranque en frío y gráficos por software como mitigación al validar en Android Studio.
Motor de texto a voz no disponible en español	Ejecución	Media	Moderado	Residual: se conserva el texto grande visible si el motor español no está disponible; TTS depende del idioma/paquetes del dispositivo. Requiere validación física por modelo.
Contraste o tamaño insuficiente para el usuario	Diseño UI	Baja	Alto	Mitigado en implementación: alto contraste, tamaño visible, grillas adaptativas, semántica accesible y TalkBack; falta una prueba de accesibilidad en varios tamaños/modelos.
Pérdida de usuarios registrados al cerrar la app	Datos	Alta	Moderado	Mitigado para modo Firebase: Auth y Realtime Database, caché persistente y datos bajo UID; sesión básica en SharedPreferences. El respaldo local es solo de demostración y no ofrece persistencia de cuenta en nube.

Definición de artefactos.
Listar y describir los artefactos o entregables que serán administrados y entregados durante el desarrollo del proyecto.

Artefacto	***Descripción***
Código Android	Módulo app en Kotlin; interfaz Compose, modelos, ViewModels, repositorios y recursos.
Firebase	Google Services, reglas de Realtime Database y conexión Authentication.
Pruebas JUnit	10 clases en src/test; reporte XML y HTML incluido (40/40 aprobadas).
APK Release	APK 1.0.0 firmado para prueba en distribucion/; huellas y verificación adjuntas.
Respuesta técnica	Formato oficial completado en DOCX y PDF; riesgos, rendimiento, restricciones y anexos.
Historial Git	El ZIP conserva el historial Git local de semanas anteriores junto con los cambios de S8; el remoto está identificado en README. Los cambios de S8 no se subieron al remoto.

Condiciones de aceptación para cierre del proyecto
Condiciones que se deben cumplir para dar termino al proyecto y margen de tolerancia de aceptación de defectos.

• La aplicación conserva las vistas Compose y el flujo con Firebase Auth/Realtime Database.
• Las alertas y perfiles tienen operaciones de persistencia asociadas al usuario; reglas de base de datos verifican UID.
• testDebugUnitTest: 40 pruebas aprobadas; sin fallos, errores ni omitidas.
• assembleRelease genera APK 1.0.0; apksigner valida firma v2/v3 y zipalign confirma alineación.
• El informe describe límites de pruebas en dispositivos y publicación; no afirma cargas a Play Console.
• El ZIP incluye fuentes, Git, respuesta oficial, informes de pruebas, guía y APK; excluye build temporales y llave privada.

Implementación técnica y evidencia de validación

La aplicación organiza la interfaz en pantallas Compose y delega estado/acciones a ViewModels. Los repositorios separan autenticación, sesión local y almacenamiento. Firebase Authentication valida la identidad; Realtime Database guarda perfiles en /users/{uid} y alertas en /alerts/{uid}/{alertId}. Las reglas firebase.database.rules.json limitan lectura y escritura al UID autenticado y validan campos básicos.

La sesión local conserva nombre, correo, uso de Firebase y preferencia de alto contraste mediante SharedPreferences en modo privado. No almacena la contraseña. Al cerrar sesión se eliminan los datos básicos de sesión; Firebase mantiene la cuenta y sus alertas.

Siguiendo la recomendación de la revisión anterior, la construcción de referencias de usuario y alertas se centralizó en el repositorio Firebase para reducir rutas repetidas:

```
private fun currentUserId(): String? =
    FirebaseBootstrap.authOrNull(appContext)?.currentUser?.uid

private fun alertsReference(uid: String? = currentUserId()): DatabaseReference? {
    if (uid.isNullOrBlank()) return null
    return FirebaseBootstrap.databaseOrNull(appContext)
        ?.getReference("alerts")?.child(uid)
}
```

La UI incluye celdas adaptativas LazyVerticalGrid, lectura del mensaje por el motor TTS del sistema y permanencia del mensaje visual. Palette extrae colores en Dispatchers.Default y usa colores de respaldo; el cálculo no ocupa el hilo de interfaz. Estas decisiones se evalúan por inspección del código y consistencia de vistas; no se midieron FPS ni memoria.

Pruebas unitarias y resultados

Comando ejecutado: ./gradlew testDebugUnitTest. Resultado: 40/40 pruebas aprobadas, 0 fallidas, 0 errores, 0 omitidas; tiempo de ejecución sumado informado por JUnit: 0.108 s. El reporte HTML y sus XML están adjuntos. Se usa JUnit 4 en app/src/test; no se declara cobertura de Espresso, Robolectric ni Firebase Test Lab.

Clases ejecutadas:

• AlertRepositoryTest: 7 pruebas, 7/7 aprobadas.

• FirebaseModelsTest: 4 pruebas, 4/4 aprobadas.

• QuickPhraseRepositoryTest: 4 pruebas, 4/4 aprobadas.

• UserRepositoryTest: 3 pruebas, 3/3 aprobadas.

• AdvancedKotlinTest: 5 pruebas, 5/5 aprobadas.

• MessageAnalyzerTest: 3 pruebas, 3/3 aprobadas.

• RegistrationValidationTest: 6 pruebas, 6/6 aprobadas.

• UserValidatorTest: 3 pruebas, 3/3 aprobadas.

• AlertsViewModelTest: 3 pruebas, 3/3 aprobadas.

• HomeViewModelTest: 2 pruebas, 2/2 aprobadas.

La cobertura incluye validación de registro, correo y contraseñas; análisis de mensaje y prioridad; modelos, frases y repositorios; y ViewModels de inicio y alertas. El ajuste esperado del conteo de palabras de “Necesito ayuda urgente” es tres, validado por el caso de análisis correspondiente.

Versionado, generación y firma

La variante release se construyó con ./gradlew assembleRelease. El manifiesto verificado declara cl.duoc.senalsegura, versionCode 8, versionName 1.0.0, minSdk 24, targetSdk 36 y compileSdk 36. Se entregó el APK firmado distribucion/SenalSegura-1.0.0-release-signed.apk (10.6 MB).

La verificación apksigner verify --verbose --print-certs confirma los esquemas v2 y v3 con una llave RSA de 4096 bits. zipalign -c -v 4 terminó correctamente. SHA-256 del APK: 9eb620f0c229a786d851e2d48958e11f41bb9811894df8c5a9a7f809cc15a949. La huella SHA-256 del certificado es cd2b7832c27dcd724388692576acb3cae9be4d3cfefc6b84c5a0576aa760e0f4. La salida íntegra está en documentacion/VERIFICACION_FIRMA_APK.txt. La llave privada temporal no se entrega; para futuras actualizaciones se debe guardar una clave de carga permanente de forma segura.

Proceso de distribución y publicación

Para la entrega, el APK firmado queda listo para instalación controlada y pruebas. Firebase App Distribution permite habilitar un grupo de testers, invitar sus correos y distribuir la versión. También se puede compartir el APK mediante un enlace privado para instalación de prueba.

El proceso de Google Play se documenta: completar ficha, política de privacidad y declaraciones de seguridad/contenido; generar un Android App Bundle firmado con una clave de carga protegida; cargarlo en una pista interna o cerrada; verificar la app con testers; resolver observaciones; solicitar revisión y publicar gradualmente tras aprobación. Debe incrementarse versionCode en cada actualización.

Alcance: el anuncio del docente del 3 de octubre indica que para esta actividad se debe explicar el proceso de Play Store y no comprar una cuenta de desarrollador. Por tanto, el informe no afirma publicación real en Google Play ni en Firebase App Distribution.

Trazabilidad con la pauta de evaluación

• 1. Front end y back end Compose/Kotlin: pantallas Login, Registro, Recuperación, Inicio, Alertas, Detalle y Backend; ViewModels, repositorios y Firebase.

• 2. Persistencia: Firebase Realtime Database, caché persistente y reglas por UID; modo local descrito como demostración.

• 3. Auth y sesión: Firebase Authentication y SharedPreferences para datos básicos y preferencias.

• 4. CRUD: crear, observar/consultar, actualizar y eliminar alertas y perfiles dentro de la ruta autenticada.

• 5. Pruebas: JUnit 4; 40 casos ejecutados con reporte XML/HTML adjunto.

• 6. APK: assembleRelease, versionCode 8 y versionName 1.0.0.

• 7. Firma: firma v2/v3, RSA 4096, apksigner y zipalign verificados.

• 8. Git: el ZIP conserva el historial local de semanas anteriores junto con los cambios de S8; el remoto está indicado en README. Los cambios de S8 no se subieron al remoto.

• 9. Documentación: formato oficial, funciones, rendimiento cualitativo, riesgos, restricciones, pruebas, versión, firma y proceso de distribución incluidos.

Los resultados son evidencias técnicas del proyecto entregado. Las pruebas de UI en dispositivos, el envío de la app a una tienda y benchmarks cuantitativos quedan fuera de esta validación y se identifican como restricciones.

Anexos

Anexo 1: Diagrama EDT

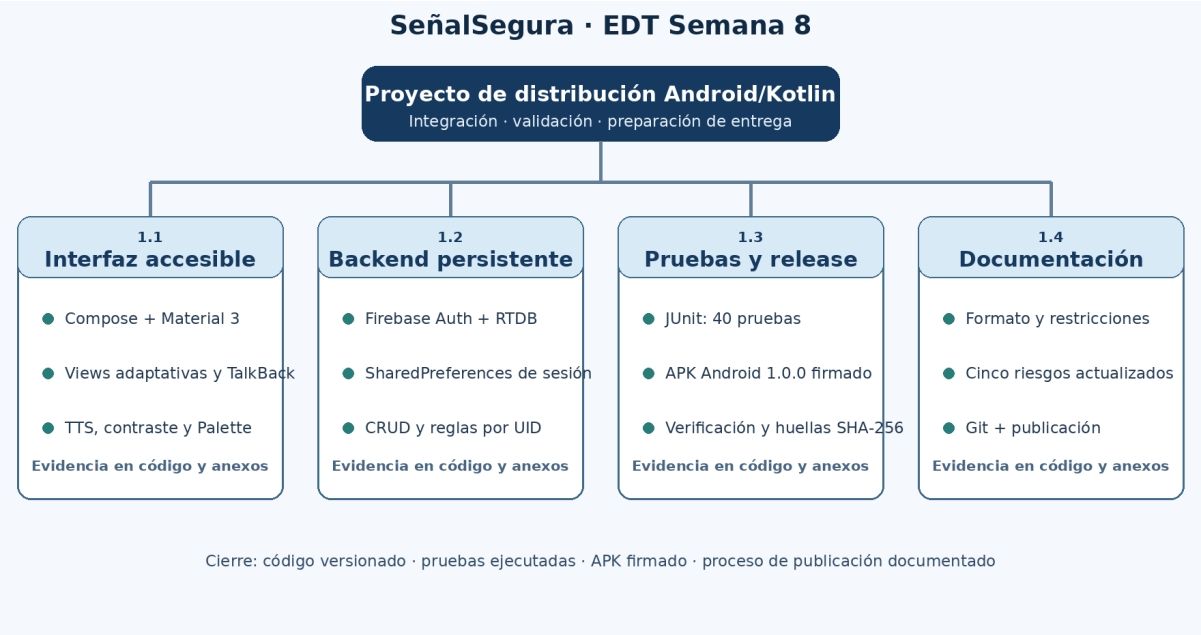


Figura 1. Estructura de desglose de trabajo de SeñalSegura para Semana 8.

Anexo 2. Diccionario EDT

1.1 Interfaz accesible: composición Material 3 con navegación, mensajes visibles, contraste y controles semánticos.
1.2 Persistencia y autenticación: Firebase Auth, Realtime Database, reglas por UID y SharedPreferences para sesión básica.
1.3 Pruebas: diez clases JUnit en src/test que verifican validadores, modelos, repositorios y ViewModels; resultado 40/40.
1.4 Preparación release: versionCode=8, versionName=1.0.0, generación APK, firma v2/v3 y verificación de alineación.
1.5 Distribución y cierre: formato oficial, guía de Play/Firebase App Distribution, cinco riesgos actualizados, README e historial Git.


DuocUC
ONLINE